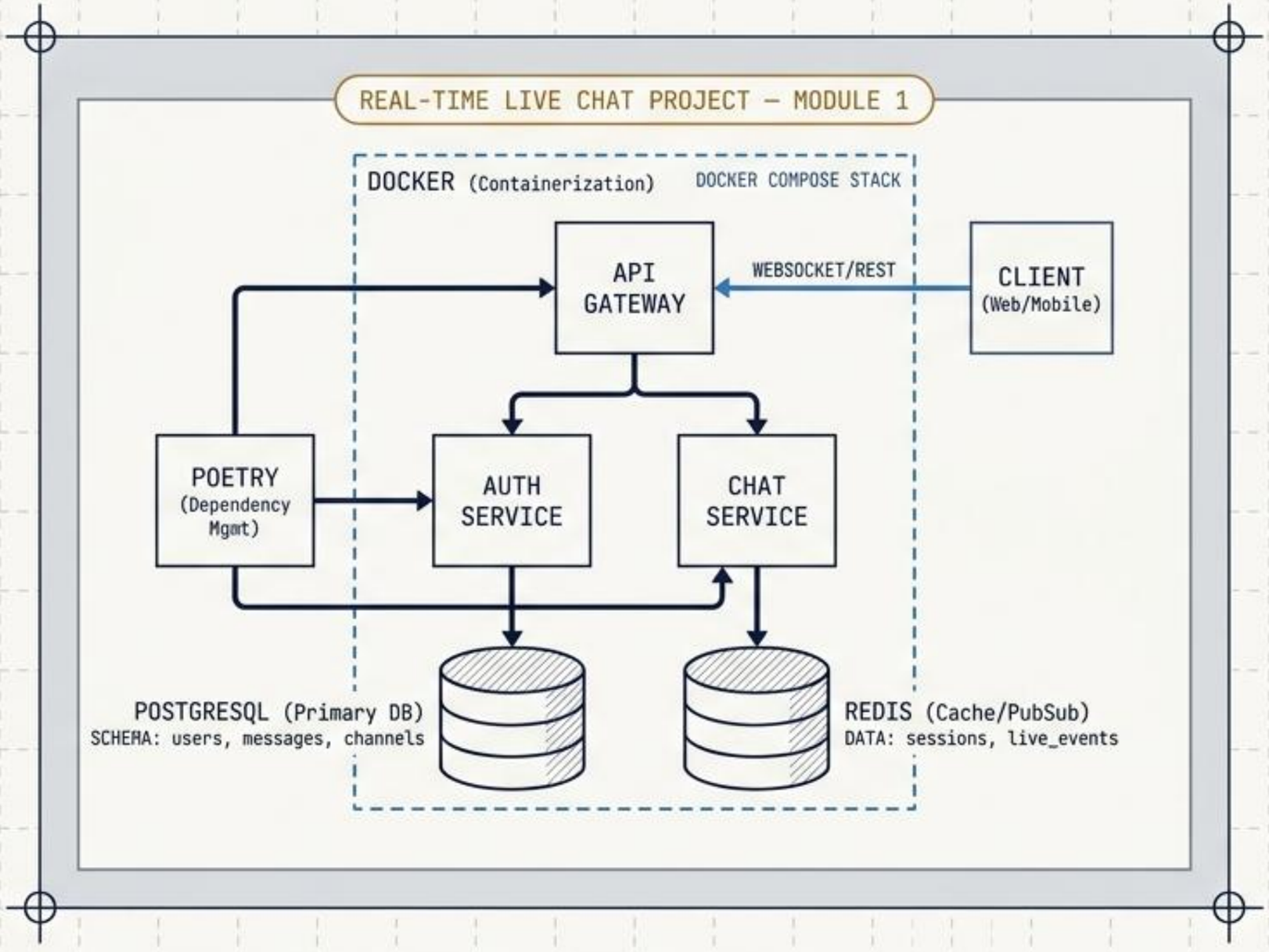


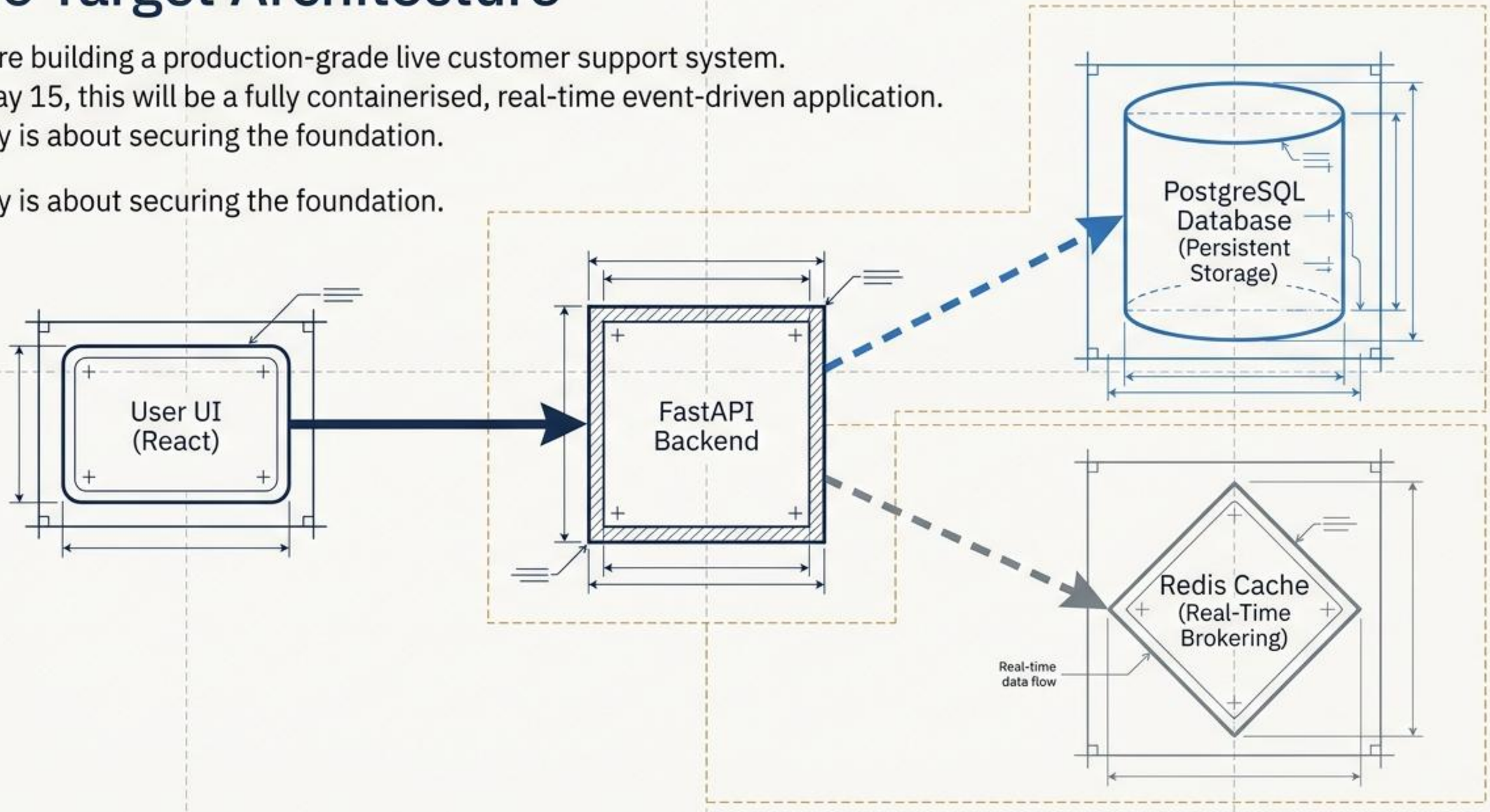
PYTHON + DB FAST-TRACK

BUILDING A BATTLE-READY FOUNDATION: ENVIRONMENT, POETRY & DOCKER

The Target Architecture

We are building a production-grade live customer support system.
By Day 15, this will be a fully containerised, real-time event-driven application.
Today is about securing the foundation.

Today is about securing the foundation.

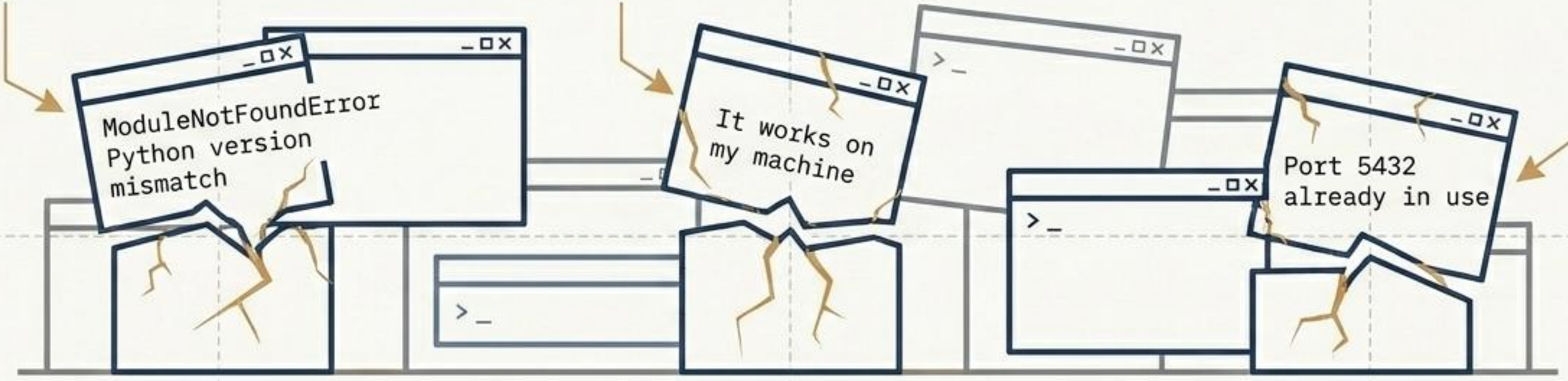


The Fragile Foundation of Local Development

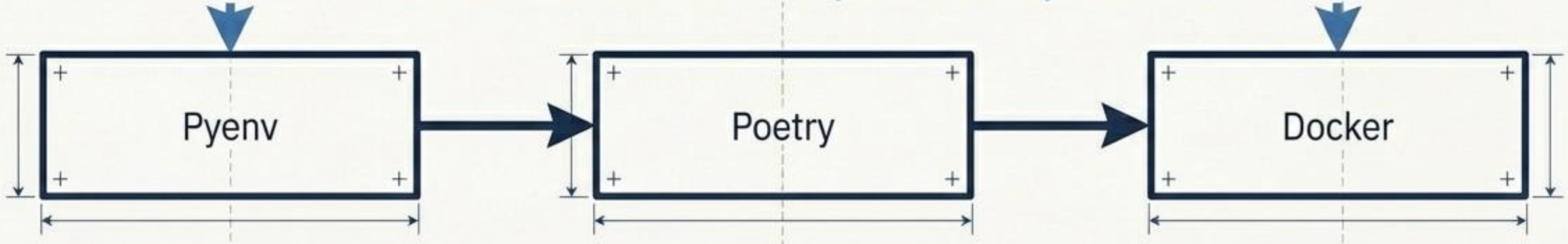
Pillar 1: OS Python is Locked
Upgrading system Python risks breaking local operating systems.

Pillar 2: Dependency Drift
Unresolved dependency trees cause severe team friction.

Pillar 3: Environment Collisions
Local database installations cause port conflicts and bleed across projects.



The Isolation Stack Solution: Pyenv → Poetry → Docker

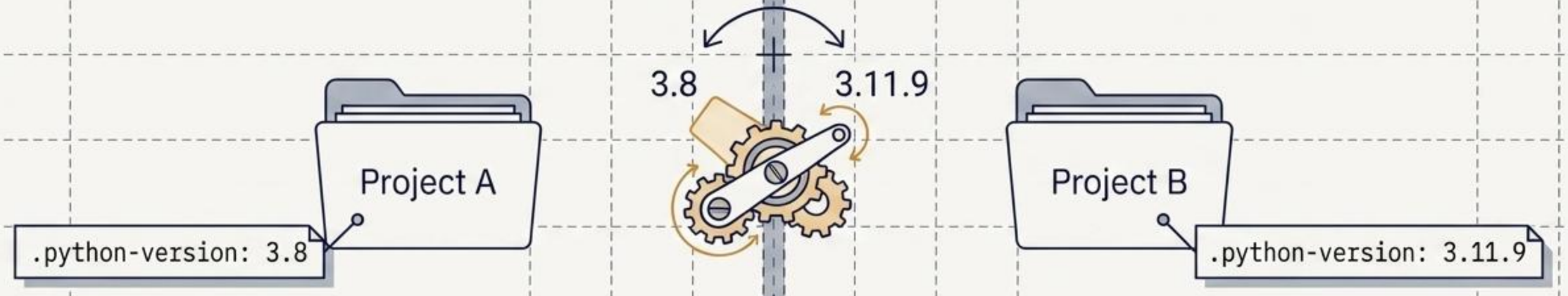


Layer 1: Isolate the Core with Pyenv

Pyenv compiles and manages multiple Python versions without touching the system OS.

Terminal Execution

```
pyenv install 3.11.9 (Downloads & compiles)
pyenv local 3.11.9 (Pins version to current
directory via .python-version)
```



Pro-Tip: Deploy with the exact same Python version you developed on.

Deterministic Dependencies: The Poetry Standard

The `poetry.lock` file is the single source of truth for CI/CD. A team of five developers will boot into the exact same dependency tree, eliminating version drift.

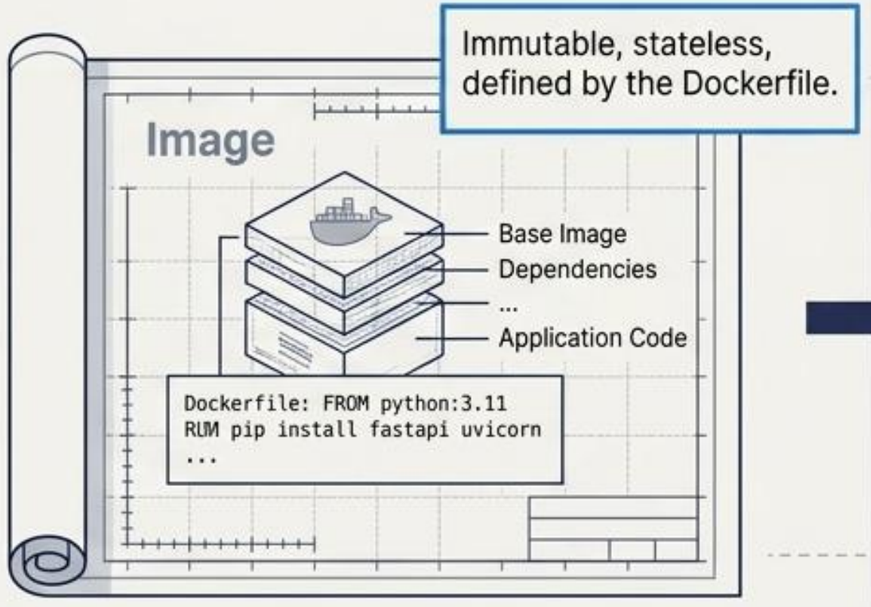
The Legacy Way	The Modern Standard
Tooling: `pip` + `venv`	Tooling: `Poetry`
Definition: `requirements.txt` (flat, fragile list)	Definition: `pyproject.toml` (rich metadata + nested dependencies)
Determinism: Works on my machine risk	Determinism: `poetry.lock` (perfect deterministic resolution)
Environment: Manual `source venv/bin/activate`	Environment: Automatic virtual environment management

The Package Lifecycle in Action

Layer 3: Reproducible Infrastructure

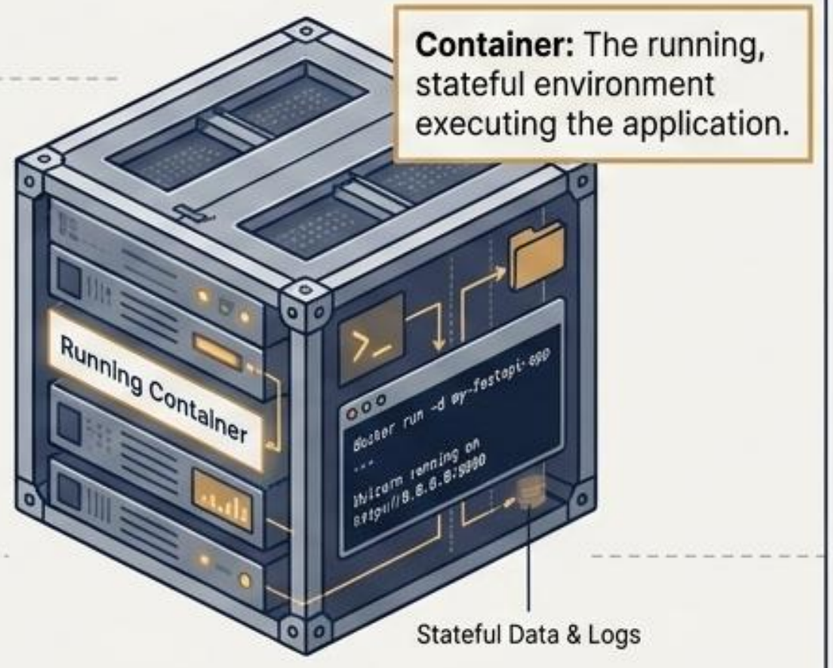
Why Docker here? We require `FastAPI`, `Postgres`, and `Redis`. Installing these natively creates environment chaos. Docker provides isolated environments that mirror production perfectly.

The Blueprint / Class



instantiation

The Object / Instance



Crafting the FastAPI Image & Layer Caching

```
FROM python:3.11-slim
```

Lightweight base; avoids Alpine C-extension issues.

```
RUN apt-get install -y libpq-dev
```

Crucial system dependency for psycopg2.

```
COPY pyproject.toml poetry.lock .
```

Copied before app code. If dependencies are unchanged, Docker caches the heavy install layer.

```
RUN poetry config virtualenvs.create false
```

Installs globally within the container; no nested venv required.

```
COPY ./app /app
```

App code is copied last, ensuring fast rebuilds on code edits.

Layer Caching pipeline

BASE LAYER: Lightweight OS (python:3.11-slim)

SYSTEM DEPENDENCIES: libpq-dev

DEPENDENCY CACHE: Project Files & Lockfile (Cached if unchanged)

Docker Caches this layer, bypassing heavy dependency install if files are identical.

CONFIGURATION: Global Poetry Config (No nested venv)

APPLICATION LAYER: App Code (Rebuilt on every code edit)

Scaffolding the API Ping

Context: This highly simplified endpoint acts as our infrastructure health-check before we introduce real-time WebSockets and database connections.

Directory Structure

```
realtime-chat/
├── app/
│   └── main.py
├── pyproject.toml
├── poetry.lock
└── Dockerfile
```

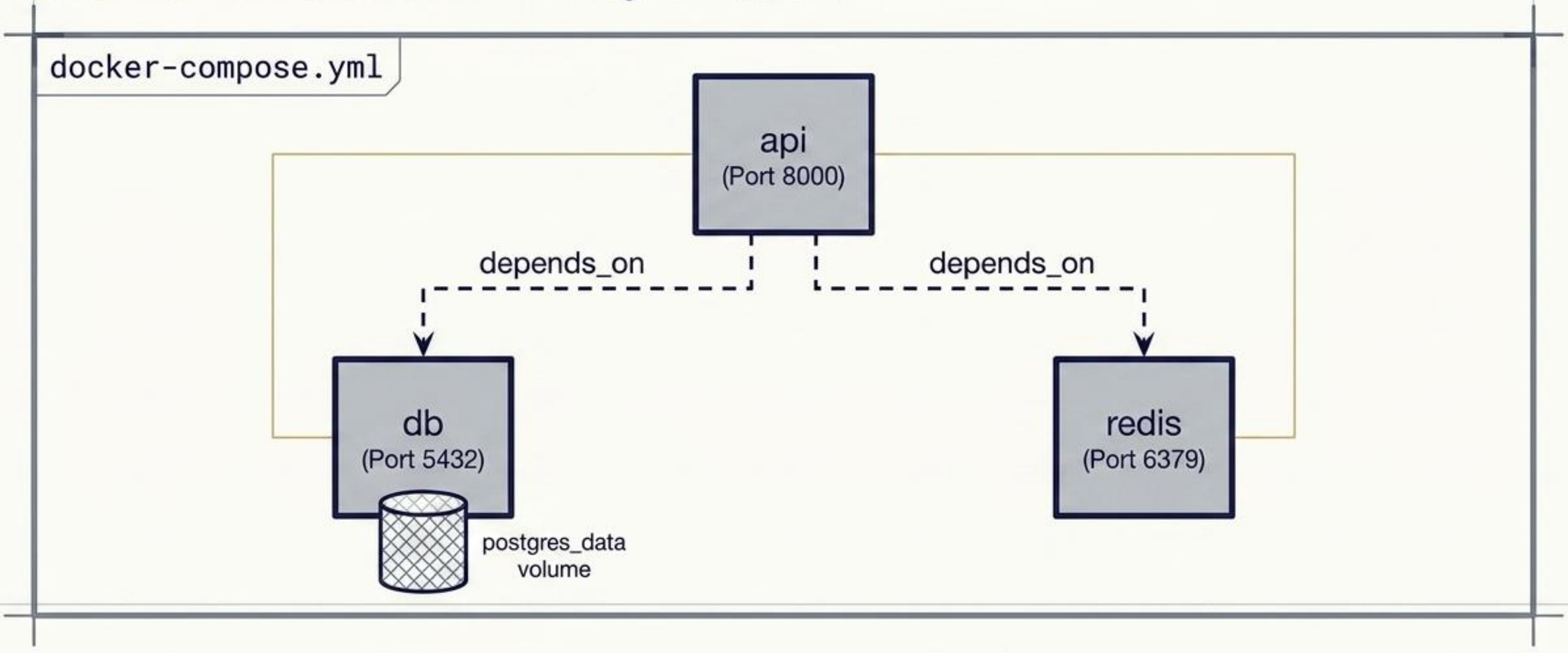
app/main.py

```python
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
async def root():
    return {'message': 'Hello from FastAPI in Docker!'}
```

```
$ curl -X GET http://localhost:8000/
{
    "message": "Hello from FastAPI in Docker!"
}
```

Layer 4: Orchestration Network

A single YAML file defines the services, internal network routing, and persistent data volumes. No manual network bridges required.



Decoding the Orchestrator

Forces the api service to build from our custom Poetry Dockerfile.

```
build: .
```

```
ports: '8000:8000'
```

Maps the host machine's port securely to the isolated container port.

Enables hot-reloading by syncing local file edits directly into the running container.

```
volumes: ./app:/app
```

```
depends_on:
```

Enforces a boot sequence; ensures DB and Redis initialize before the API attempts a connection.

The 4-Line Synthesis Bootstrap

You have successfully bootstrapped a battle-tested, deterministic microservice environment in seconds.

```
1. pyenv local 3.11.9
```

Your Python version is locked.

```
2. poetry install
```

Your dependency tree is locked.

```
3. docker compose up --build -d
```

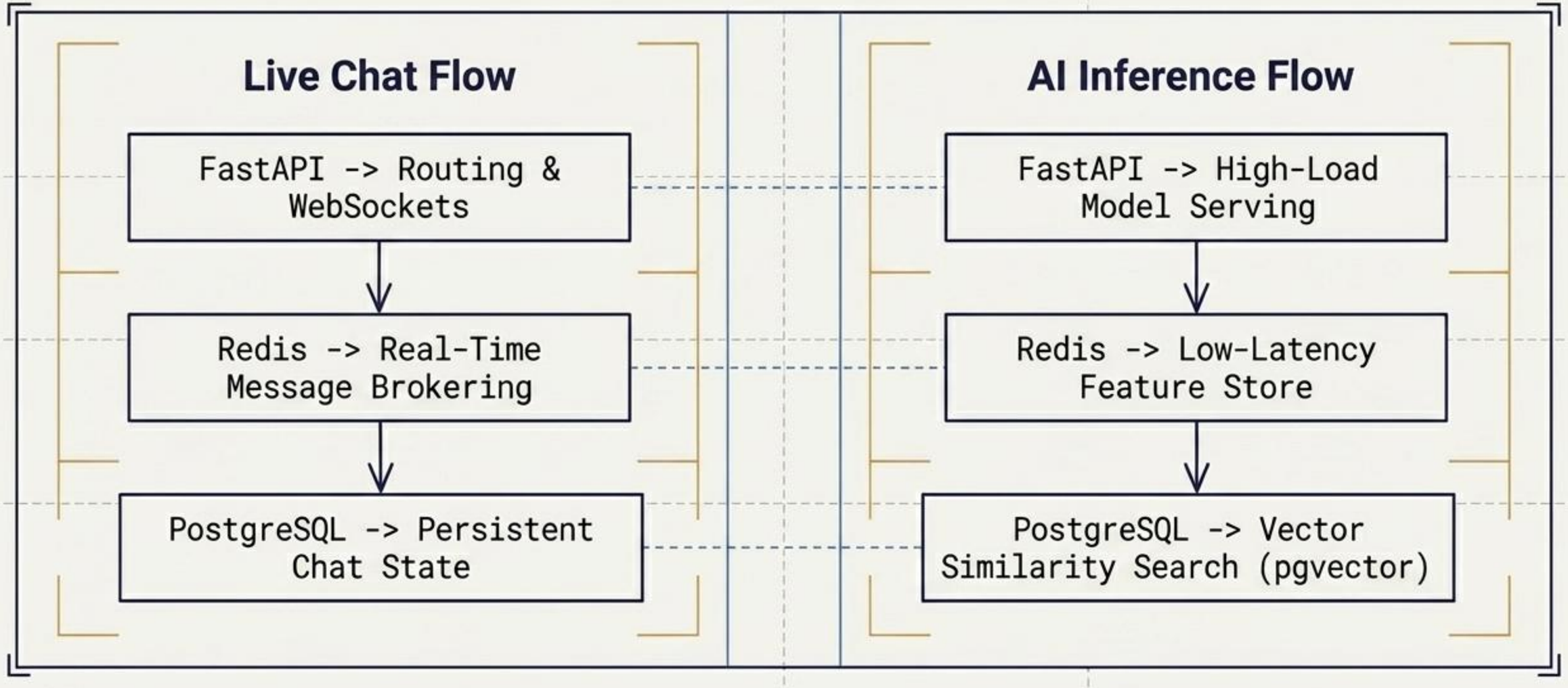
Your infrastructure and networking are locked.

```
4. curl http://localhost:8000/
```

System verified.

Stack Symmetry: The AI & Enterprise Lodiec Link

You are not just building a chat app; you are mastering the exact infrastructure used by leading AI-driven startups.



Diagnostic Matrix: Day-1 Roadblocks

You has a three thao matesting slids, with lombiles, running footels, numbers, or breadcrumbs.

Symptom	Diagnosis	Fix
Port 5432 already in use.	Local host Postgres instance conflicting with Docker.	Stop local Postgres, or remap compose ports to `5433:5432`.
Code changes not reflecting in browser.	Container running statically despite volume mount.	Ensure volume `./app:/app` is set and restart via `docker compose restart api`.
psycopg2 failing to build during poetry install.	Missing C-compiler libraries in slim image.	Ensure `apt-get install libpq-dev` is present in the Dockerfile.

Module 1 Assignment & Next Steps

Pre-Flight Checklist

- [ ] Verify docker compose up successfully returns the FastAPI 'Hello' message.
- [ ] Modify main.py to add a /health endpoint returning {'status': 'ok'}. Rebuild and test.
- [ ] Bonus: Implement a .env file for DB credentials and reference them via ${VARIABLE} syntax in docker-compose.yml.

Preview: Module 2: FastAPI Foundations

Coming up next: Async handlers, Pydantic data models, and REST endpoints. Keep your terminals ready.